Maren

CONFIDENTIAL

Maren Finance

Testnet Build, Complete

Everything built so far, the evidence for each claim, and an honest list of what is still broken. Every number here is one you can reproduce yourself.

DOCUMENT	DATE	VERSION	PREPARED FOR
MRN-101	16 September 2026	1.0	Mezo Ecosystem

DOCUMENT	DATE	CLASSIFICATION	VERSION
MRN-101	16 September 2026	Confidential	1.0

Maren Finance — Testnet Build, Complete

Everything built so far, the evidence for each claim, and an honest list of what is still broken. Every number here is one you can reproduce yourself.

CONTENTS

1 What you can check yourself, in five minutes 2

2 What Maren is 2

3 Pilot metrics, read live on 16 September 3

4 What is actually built 4

5 Proof of work: the engineering discipline 4

5.1 Deployed bytecode beats documentation 5

5.2 We audited ourselves adversarially, and published what it found 5

5.3 Honest data, enforced rather than intended 6

6 Where this honestly stands 6

7 Open issues, stated plainly 7

8 Infrastructure, briefly 8

9 Try it 8

10 What the grant would fund 8

1

What you can check yourself, in five minutes

Site	`https://marenfinance.xyz`
Docs	`https://marenfinance.xyz/docs`
iOS beta	`https://testflight.apple.com/join/uhmv9YPE`
Relayer health	`https://maren-relayer.rolandojude18.workers.dev/healthz`
Chain	Mezo **matsnet**, chain ID 31611

Five contracts, all source-verified on Blockscout:

CONTRACT	ADDRESS
MarenVault	`0x46252B00AB59384518d9dB6D1f210ffF1221645b`
MarenPay	`0xe475Ff919D54E055809f516E3bed8a5c7a101836`
MarenEarn	`0xE3B02b215085B99Ce61E0F5ea9763Ac637aa7826`
MarenRegistry	`0x9e123Cdd0CD524812CF9F552d3FAB13d8143b77E`
MarenRelayHub	`0x22cC1Adfb0C91D9359746dE3F7E8cB7F2B8eA98C`

The pooled trove is real, and it lives inside Mezo's own TroveManager:

```bash
cast call 0xE47c80e8c23f6B4A1aE41c34837a0599D5D16bb0 \
  "getTroveColl(address)(uint256)" \
  0x46252B00AB59384518d9dB6D1f210ffF1221645b \
  --rpc-url https://rpc.test.mezo.org
```

2

What Maren is

A person holding Bitcoin who needs cash has one option: sell it. Maren gives them a second — deposit BTC, borrow MUSD against it, and spend that MUSD, without selling a satoshi.

It is a consumer client for Mezo's own MUSD lending protocol. Maren does not re-implement lending. It makes Mezo's lending usable by someone who does not know what a trove is.

The problem it solves for Mezo specifically. MUSD's protocol minimum is 1,800 MUSD of net debt — about **$5,000 of Bitcoin** at the 250% collateral ratio Maren recommends. Read on mainnet today: MUSD's system debt sits across **19 open troves**. Borrowing is whale-shaped because the minimum makes it whale-shaped.

Maren's pooled vault removes that minimum. Its own floor is `MIN_DEPOSIT = 0.001 BTC` — about $76 at today's price — enforced in `MarenVault._requireDepositAllowed`. There is no minimum *loan* at all: the only amount check on the borrow path is `amount == 0`, and what actually binds is the 180% collateral ratio. A depositor gets a proportional claim inside one conservatively managed trove that clears MUSD's 1,800 minimum on their behalf. That is the entire product thesis, and it is the part that required original contract work rather than integration work.

3

Pilot metrics, read live on 16 September

Regenerate every figure below with `pnpm --filter @maren/metrics start`. It reads every Maren contract from the deployment block — 14,911,471, located by bisecting `eth_getCode` — through to chain head.

Funded positions	**6**
MUSD borrowed through Maren	**2,944.72** gross, all time
Payments settled	**17**, totalling 82.70 MUSD
Earn TVL	**54.00 MUSD** across 4 depositors
Handles registered	2
Money links created / claimed	1 / 0
Spend ratio	**80.0%** — 4 of the 5 borrowers went on to send a payment

The spend ratio is the number that matters. Maren's thesis is that Bitcoin-backed credit creates *spending*, not merely borrowing. Four of the five addresses that borrowed then paid someone.

These are testnet figures from a handful of accounts, most of them ours or our two beta testers'. They demonstrate that the loop works end to end. They are not evidence of demand, and this document will not present them as such.

4

What is actually built

Five contracts on matsnet, all source-verified: the pooled vault, payments, an ERC-4626 Earn vault over MUSD's Stability Pool, a handle registry, and an ERC-2771 relay hub.

A native SwiftUI iOS app, live on TestFlight at build 66. Onboarding generates a secp256k1 key on-device behind Face ID — there is no account, no server-side login, nothing registered with us. It ships borrow, repay, deposit, withdraw, pay by `@handle`, claimable links, split payments, payment requests, Earn, and an activity history read from chain logs.

A hosted gasless relayer on Cloudflare Workers, with a Durable Object for nonce sequencing and per-user quota. It is fail-closed: it simulates every call before paying, and declines rather than burning gas on something that would revert.

Proof, not a claim: a real ERC-2771 sponsored transaction settled in **block 15368268** — gas paid by `0xd902…df89` on behalf of a signer holding **zero BTC**.

A TypeScript SDK over viem, shared by the web client and the metrics tool, plus a marketing site and a twenty-page documentation set.

5

Proof of work: the engineering discipline

Three things distinguish this from a demo, and all three are checkable.

5.1

Deployed bytecode beats documentation

Mezo's published documentation contradicts its deployed contracts in places. Every chain fact Maren relies on was verified against the implementation bytecode — read through the EIP-1967 proxy slot — rather than taken from a documentation page.

Result: zero selector defects. All 25 hand-written iOS ABI encoders, every SDK ABI fragment, and every Solidity interface are byte-identical to Mezo's deployed dispatch table. Each was run through `cast sig` and the four resulting bytes searched for in the runtime code. Negative controls confirm the divergences we documented are real: the Liquity v1 forms `openTrove(uint256,uint256,address,address)`, `provideToSP(uint256,address)` and `lastGoodPrice()` are all **absent** from Mezo's deployed code, exactly as our constants claim.

Three findings are worth passing back to the Mezo team.

Pyth's Mezo feeds are dead on both networks. Measured 7 September: BTC/USD `publishTime` was **41.9 days** stale on testnet and 51.5 days on mainnet, returning **$65,238** against a live $79,466 — an 18% error. `getPriceNoOlderThan(BTC/USD, 3600)` reverts `StalePrice` on both networks. The documentation presents these fourteen feeds as usable; an integrator following that page would put a 20%-wrong price into a transaction path.

`developers/chains/index.md` is wrong for every EVM developer. It gives Cosmos chain IDs where `configure-environment.md` correctly gives 31612 and 31611, and it lists three hostnames that do not resolve.

The 60-second oracle staleness revert is undocumented, and it is the single most important MUSD integration constraint. `PriceFeed.fetchPrice()` reverts once the oracle is more than 60 seconds stale, which fails every borrow, adjust, liquidate and redeem path for reasons entirely unrelated to the caller.

5.2

We audited ourselves adversarially, and published what it found

Four parallel audits ran against `mezo-org/documentation`, `mezo-org/musd`, and the thirteen PDFs in `mezo-org/audits`. Five more ran against our own documentation, with instructions to disprove every claim on the site. They found real defects in our code. The ones still open are listed below, in full.

One example of what that caught. Our borrow screen computed the maximum borrow from Maren's own 180% collateral floor and ignored Mezo's per-trove `maxBorrowingCapacity`, which is fixed at the price the trove opened at. Measured against the live trove, the app offered **1,319.10 MUSD** where the real limit was **84.52**. Confirmed by simulation: 84 clears, 90 reverts with *"BorrowerOps: An operation that exceeds maxBorrowingCapacity is not permitted."* A beta tester hit exactly this. It is fixed on both clients, with three regression tests pinned to the measured figures.

5.3

Honest data, enforced rather than intended

The app never fabricates a number. If a value cannot be read, it says so. A transaction is reported successful only on a receipt, never on a hash.

This is enforced, not merely intended: CI runs `strings` over the built Release binary and fails if any fabricated-data symbol survives — because `#Preview` is **not** `DEBUG`-only, and was quietly linking a stub client into release builds.

The same rule applies to the documentation. Our own audit found, and we then corrected, a page describing a redemption mechanism that is not in the deployed contract, a claim of "multisig and timelock" when a single key holds every role, and a "strict CSP" that did not exist. Those corrections are live on the docs site now.

6

Where this honestly stands

Testnet only. Mainnet addresses are all zeroes, and no code path reaches a production network. No real funds are at risk.

Pre-audit. No third-party security audit of Maren's contracts exists. A paid audit of MarenVault, MarenEarn and MarenRelayHub is the gate on mainnet, and that ordering is deliberate: the pooled vault is the component that can lose user funds through a Maren design failure, and it should not hold real money before someone independent has read it.

Against Mezo's own `dapp-requirements`, we satisfy one of three. MUSD integration, yes. Third-party audit, no. Mainnet deployment, no. Both gaps are stated on our public risk page.

7

Open issues, stated plainly

These are in our tracker and on the public risk page. None is hidden.

The pooled trove is at its borrowing ceiling. Verified today: a ceiling of 2,749.20 MUSD against 2,749.13 of debt — **0.07 MUSD of headroom**. Borrowing is effectively blocked until the trove is refinanced. The trove itself is healthy at a 190.6% collateral ratio on 0.069061 BTC; it is the capacity cap that binds, not risk. The app reports this honestly rather than offering an amount that would revert. *Open, operational.*

`getEntireDebtAndColl` **pending values are double-counted** in four places. Mezo's TroveManager folds `pending*` into its first three return values before returning them; we then add them again. On the first liquidation the Stability Pool cannot absorb, every pooled user would be charged twice the redistributed debt. Mezo's DefaultPool is 0/0 today, so this is armed and not firing. *Open, critical.*

The deployed vault predates its own redemption reconciliation. `reconcileRedemption`, `settleRedemption`, `collateralOf` and `redemptionShortfall` are absent from the deployed dispatch table — confirmed again today. The mechanism exists in source and in the test suite; it is not on chain. *Open, needs redeploy.*

A liquidation of the pooled trove freezes every exit, including idle BTC that never entered the trove, with no admin recovery path. Disclosed on the risk page as R-10. *Open, by design pending a wind-down mechanism.*

Earn can lose principal. Cantina's mUSD audit §3.4.7 established that Stability Pool liquidations can be unprofitable; Thesis agreed and marked it **Acknowledged, not fixed**. Maren is structurally the passive depositor in that scenario. Disclosed as R-11. *Disclosed, not mitigable by us.*

One EOA holds every privileged role on matsnet — admin, keeper and guardian. A Safe with a timelock is a hard mainnet gate. Disclosed as R-13. *Open.*

The keeper is not built. Liquidation functions are deployed and `KEEPER_ROLE`-gated, but no automated process calls them. *Open.*

The repository is private during grant review. Access can be granted immediately on request. *Open.*

8

Infrastructure, briefly

iOS builds run on **Xcode Cloud**, which always carries the current Xcode — Apple now requires the iOS 26 SDK for uploads, and our development machine runs Xcode 16.2. The build generates its `.xcodeproj` from `project.yml` in a post-clone hook rather than committing it, and installs SwiftPM dependencies from a committed `Package.resolved` pinning all 23 transitive packages, so CI and local builds are provably identical.

The web client deploys to Vercel. The relayer runs on Cloudflare Workers.

9

Try it

The iOS beta is at `https://testflight.apple.com/join/uhmv9YPE`. There is no sign-in; onboarding takes about thirty seconds.

You will need testnet BTC — send an address and it will be funded within minutes. The first deposit needs a small amount for gas and cannot be sponsored, because that transaction carries value. Everything after it is fee-sponsored.

One thing to expect: **borrowing will currently refuse**, because the pooled trove is at its ceiling as described above. Deposit, Earn, payments, handles and links all work. It is better to know that in advance than to have it look like a bug.

10

What the grant would fund

1. **A third-party audit** of MarenVault, MarenEarn and MarenRelayHub. Everything else is downstream of this.
2. **Redeploying the vault** with redemption reconciliation present and the pending double-count fixed.
3. **The keeper**, automated first and decentralised after — today it is a single team-held key.
4. **Mainnet behind a staged cap** — 5 BTC, then 25, then 100, each step gated on clean operating history rather than on a date.

Every chain figure in this document was read live from Mezo RPCs on 16 September 2026 and is paired with the command, address, or transaction hash that reproduces it. The machine-readable stamp is `docs/src/live.json`; regenerate it with `node tools/docs/stamp-live.mjs`.

Where Mezo's published documentation and the deployed bytecode disagree, this document follows the bytecode and says so. Those disagreements are offered to the Mezo team as findings, not as criticism.